Nirmol — System Architecture

Oct 18, 2026 · @Mrittika Jahan

Summary

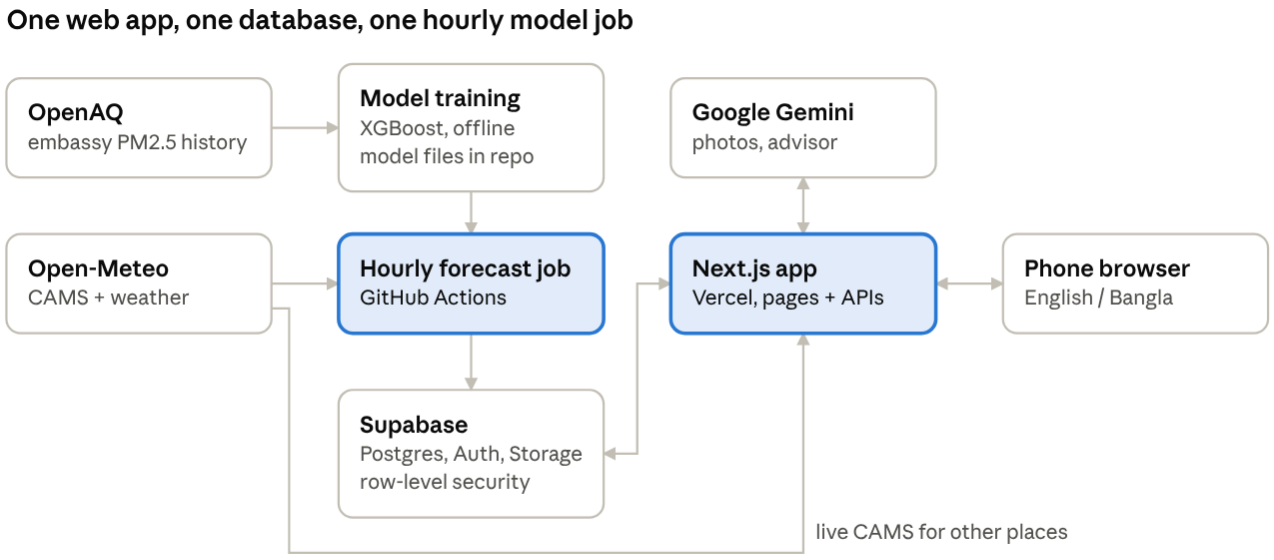
Nirmol is a bilingual (English/Bangla) air-quality platform for Bangladesh that forecasts PM2.5, turns it into personal decisions, and lets citizens report pollution sources. It is designed as a **mobile app**; within the hackathon time limit it was built as a **mobile-first web app** whose screens and backend carry over to a native or installable app.

Key design choices:

- **One Next.js application** (pages + API routes) on Vercel, instead of separate frontend and backend servers.
- **Supabase** (PostgreSQL + PostGIS, Auth, Storage) for accounts, reports and forecasts, with row-level security on every table.
- **An offline-trained XGBoost model** that corrects the CAMS global forecast for Dhaka, run every hour by a **GitHub Actions** job that publishes to Supabase.
- **Gemini** for photo-based source suggestions and the Bangla/English advisor, constrained to fixed categories and reviewed advice so it cannot invent numbers.
- **Honest data labelling**: every value carries its source (model estimate, measured, demo) and freshness; missing data is never shown as zero.

The whole system runs on free tiers. Source code: `github.com/Mrittika150/Nirmol`.

Architecture at a glance



Nirmol architecture · data sources, model job, database, web app

The model is trained once offline; every hour the GitHub Actions job runs it on live Open-Meteo inputs and publishes the Dhaka forecast to Supabase. The Next.js app reads that forecast, fetches live CAMS for all other places, stores accounts and reports in Supabase, and calls Gemini only from the server.

Components and technology stack

Layer	Technology	Role
Web app (UI)	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS 4	Mobile-first pages, English/Bangla text, light/dark themes
Charts and maps	Recharts; Leaflet with a bundled Bangladesh outline (geoBoundaries)	Forecast chart with range band; Community Map
API layer	Next.js route handlers (server-side)	Air data, reports, moderation, advisor, auth, account
Database	Supabase PostgreSQL + PostGIS	Profiles, forecasts, reports, reviews, quotas, account settings
Authentication	Supabase Auth (email + password, Google OAuth), PKCE via `@supabase/ssr`	Sign-in sessions in HTTP-only cookies

Layer	Technology	Role
File storage	Supabase Storage, private bucket `nirmol-report-images`	Report photos (JPEG, max 2 MiB after processing)
Image processing	sharp	Strips metadata (EXIF/GPS), resizes, re-encodes photos
Forecast model	Python, pandas, XGBoost (quantile regression), built-in SHAP	CAMS-correction model with 80% range and per-hour explanations
Scheduled job	GitHub Actions (hourly), Python 3.14 with hash-pinned dependencies	Runs the model on live inputs and publishes to Supabase
AI services	Google Gemini (`gemini-3.1-flash-lite`)	Photo source suggestion; advisor plan selection
Environmental data	Open-Meteo (CAMS air quality, weather forecasts); OpenAQ (historical training data)	Live inputs and training data
Hosting	Vercel (web), Supabase (Singapore region)	Public deployment

Data and machine learning

The forecast model corrects the CAMS global model for Dhaka and is 36% more accurate than raw CAMS on a held-out winter.

Why a correction model. A station audit of OpenAQ found 23 PM2.5 locations in Bangladesh; the only reference-grade monitor (US Embassy, Baridhara) stopped publishing on 24 March 2025. A model that needs live past measurements therefore cannot run today. Instead, the model predicts the embassy's measured PM2.5 from inputs that are available live: the CAMS PM2.5 forecast, weather forecasts and calendar features (model output statistics).

Training data (`jobs/build_training_data.py`): 22,872 hourly rows from 15 Aug 2022 to 24 Mar 2025, joining embassy PM2.5 (OpenAQ), CAMS PM2.5/PM10/CO/dust (Open-Meteo) and archived weather forecasts; 90% of hours have a measurement. Winter (Nov–Feb) means: measured 170 µg/m³, CAMS 81 µg/m³.

Model (`ml/train_forecast.py`): three XGBoost quantile regressors (10th, 50th, 90th percentile) on log1p(PM2.5), with 24 features: CAMS values and their 24-hour mean, temperature, humidity, dew point, rain, pressure and its 24-hour change, cloud, wind

speed and direction (u/v), sunshine, hour, month (cyclic), Friday–Saturday weekend and winter flag. Train: Aug 2022 – Oct 2024. Test (never seen): Nov 2024 – Mar 2025. The 80% range is calibrated with conformalized quantile regression on Nov 2023 – Oct 2024.

Results on the held-out test period (µg/m³):

Method	MAE	RMSE	Usable live?
Nirmol model	51.1	76.6	Yes
Raw CAMS	80.3	108.6	Yes
Persistence, 24 h	61.4	95.2	No (needs a live monitor)
Persistence, 1 h	26.4	46.5	No

Winter hours only: 53.5 vs 84.2 (CAMS). Range coverage: 81% (target 80%). Most influential inputs after CAMS itself: temperature (cold nights trap smog), CAMS 24-hour mean and dew point. Full report: `docs/model_evaluation.md`.

Hourly forecast pipeline (`jobs/refresh_forecast.py`, GitHub Actions):

1. Fetch the latest CAMS and weather forecasts for Baridhara from Open-Meteo, with 2 days of history for rolling features.
2. Build the same features as training and run the three saved models for the next 25 hours.
3. Compute the top 3 SHAP contributions per hour from XGBoost's built-in tree SHAP.
4. Validate every value, then publish the run atomically through the `publish_nirmol_forecast` database function (idempotent: a retry never creates a duplicate).
5. The web app reads the latest run, re-validates it, and uses it only if it is fresh and complete; otherwise it falls back to CAMS.

The Nirmol model is shown only at the evaluated Baridhara point; all other locations show CAMS with a warning that it underestimates winter pollution.

Application layer

Public tools work without an account; only reporting, moderation and saved settings need sign-in.

Pages

Page	Feature	Data used
`/`	Dashboard: live PM2.5, AQI estimate, weather, 24 h forecast, "why" factors	`/api/air`
`/exposure`	Exposure score = PM2.5 × hours × activity factor (1.0 / 1.5 / 2.5), stricter levels for sensitive groups	`/api/air`
`/travel`	Best departure time within a window (minute-by-minute over the hourly forecast)	`/api/air`
`/simulator`	Compare two scenarios	`/api/air`
`/alerts`	In-app threshold alerts	`/api/air`
`/diary`	Exposure and symptom log in browser storage only	`/api/air`
`/reports`	Submit and manage pollution reports	`/api/reports/*`
`/moderation`	Review queue for moderators	`/api/reports/moderation/*`
`/community`	Map of approved reports	`/api/reports/public`
`/advisor`	Bilingual AI advisor	`/api/advisor`
`/login`, `/my-nirmol`	Email/Google sign-in, saved places and preferences	`/api/auth`, `/auth/google`, `/auth/callback`, `/api/account`
`/guide`	How to use Nirmol	—

Key API routes (server-side only)

- `GET /api/air?lat&lon`: validates the point is inside Bangladesh, fetches CAMS and weather from Open-Meteo, reads the latest Nirmol run from Supabase at Baridhara, and returns values with source, issue and fetch times. Malformed, stale or contradictory provider data is rejected rather than shown.
- `POST /api/reports`, `/api/reports/images`, `/api/reports/classify`: create a report, upload and normalise a photo, and request an optional Gemini suggestion limited to 7 categories (waste burning, construction dust, vehicle smoke, industrial smoke, brick kiln, other, uncertain).

- `/api/reports/moderation/*`: moderator queue, detail and approve/reject with a reason; `/api/reports/public`: approved reports as 0.05° (~5 km) cells only.
- `POST /api/advisor`: checks the daily quota, builds verified facts from `/api/air`, asks Gemini to choose up to 3 reviewed guidance codes and fact IDs (structured JSON), then renders the answer from Nirmol's own text. Emergency phrases in English, Bangla and Banglish are answered directly without calling Gemini.

Database, security and privacy

Every table has row-level security, private data lives in schemas the public API cannot reach, and secret keys never reach the browser.

Schema (`supabase/migrations/0001` – `0006`):

Area	Tables / objects	Notes
Accounts	`profiles`, `account_settings`	Language, saved places, `is_moderator` flag
Forecasts	`forecast_runs`, `forecast_points`, `publish_nirmol_forecast()`	One immutable run per hour, 25 points with range and SHAP factors
Reports	`pollution_reports`, `report_reviews`, `nirmol_reporting.*` (images, classifications, budgets)	Exact location stored privately; public view uses coarse cells
Advisor	`nirmol_advisor.counts`, `attempts`	Daily quotas only; chat text is never stored

Security controls

- **Row-level security** on all public tables: users read and write only their own rows; moderators are checked by a security-definer function.
- **No self-promotion:** a trigger blocks users from setting `is_moderator`; moderators are assigned only through admin SQL.
- **Private schemas:** `nirmol_reporting` and `nirmol_advisor` are not exposed through the REST API; access goes through guarded database functions.
- **Private photo bucket:** JPEG only, 2 MiB limit; restrictive storage policies block direct access, and photos are served only to the owner or a moderator via server routes.
- **Authentication:** PKCE code exchange for email links and Google sign-in, HTTP-only cookies, exact-origin checks on every write, and an allow-list for redirects.

- **Secret handling:** the service-role key, Gemini key and advisor cookie secret are used only on the server.
- **Abuse limits:** daily caps on reports, photo uploads, AI classifications and advisor questions (20 per user or guest per day, plus a project-wide cap).

Privacy by design

- Photos are re-encoded with metadata (including GPS tags) removed.
- The public map shows only type, date and a ~5 km cell; reporter identity, notes and photos are never public.
- The diary and alert settings stay in the browser and are never uploaded.
- The advisor sends Gemini only the question, language, verified air facts and an optional activity, never coordinates, names, account data or history.

Deployment, operations and testing

Deployment

- **Web:** Vercel builds the `web/` folder on every push to `main`. Configuration comes from environment variables (Supabase URL and keys, `NIRMOL_SITE_URL`, Gemini key and model names, feature switches, advisor cookie secret).
- **Database:** a hosted Supabase project in Singapore; the migrations are applied in order from `supabase/setup/` (step 1–7), with a check script.
- **Forecast job:** `.github/workflows/forecast.yml` runs every hour at :05, gated by the repository variable `NIRMOL_ATOMIC_PUBLISHER_ENABLED`; it installs hash-pinned packages, runs the Python tests and a native model check, then publishes.
- **Failure behaviour:** if the hourly job, Supabase or Open-Meteo is down, the dashboard falls back to CAMS or shows a clear "try again" message; it never shows stale numbers as current.

Testing

Suite	Count	What it covers
Web unit and integration tests (`node --test`)	451	Air-data validation, AQI, exposure and travel maths, reports, moderation, advisor safety, auth, accounts
Browser tests (Playwright)	330	Real user journeys on all pages, mobile layout, accessibility (axe)
Python tests	55	Feature building, model runtime, forecast publication rules

Plus TypeScript type-checking, ESLint with zero warnings, and a dependency audit with no production vulnerabilities.

Limitations and future work

Current limitations

- Web app, not yet a native mobile app; no push, SMS or Telegram alerts.
- The Nirmol model is validated at one site (Baridhara, Dhaka); elsewhere the app uses CAMS, which underestimates winter pollution.
- Test scores use near-real-time CAMS values, so live 24-hour-ahead accuracy will be somewhat lower.
- No live reference monitor exists to keep checking the model; low-cost AirGradient sensors are not yet used as inputs.
- Route comparison is not built: model resolution (~45 km) cannot tell nearby streets apart.
- The Gemini free tier can be busy; the advisor then falls back to a safe message.

Future work

1. Package as a mobile app (PWA first, then native) with push notifications and offline last-known data.
2. SMS and Telegram alerts in Bangla for users without smartphones.
3. Calibrated low-cost sensor network for street-level measurements and continuous model validation.
4. Extend the correction model to Chattogram, Sylhet, Rajshahi and other cities as data allows.
5. Route comparison once spatial data supports it; school, workplace and authority dashboards with exported hotspot reports.

6. Report follow-up (forwarded to authority, resolved) and evaluation of the AI photo classifier on a labelled Bangladesh image set.